

Workshop 02 – Besprechung der Lösung

Musterlösung: `loesung.py`.

Überblick

Das Skript hat zwei Modi, gesteuert über `argparse`:

- ohne Flag: Daten laden, Modell bauen, trainieren, Plot speichern
- `--predict`: gespeichertes Modell laden und ein zufälliges Testbild klassifizieren

Alle Hyperparameter stehen als Konstanten am Anfang der Datei (`EPOCHS = 50`, `BATCH_SIZE = 64`, `LEARNING_RATE = 0.0005`, `DROPOUT = 0.25`, `STEP_DROP = 5`).

1. Datenvorbereitung

Zusätzlich zur Normalisierung macht die Lösung zwei Dinge:

- **Data Augmentation** (Bonus): jedes Trainingsbild wird zufällig rotiert und horizontal gespiegelt; die Varianten werden an das Trainingsset angehängt. Das vervierfacht die Datenmenge und verringert Overfitting.
- **Zwischenspeichern** als `.npy` (`save_dataset` / `load_dataset`), damit der `--predict`-Modus die Testdaten ohne Keras-Download laden kann.

2. Architektur

Drei Conv-Blöcke mit je zwei Conv2D-Layern ($32 \rightarrow 64 \rightarrow 128$ Filter), jeweils:

```
Conv2D(n, (3, 3), padding="same", activation="relu"),
BatchNormalization(),
Conv2D(n, (3, 3), activation="relu"),
BatchNormalization(),
MaxPooling2D(2, 2),
Dropout(DROPOUT),
```

- `BatchNormalization` nach jedem Conv-Layer stabilisiert die Aktivierungen und erlaubt höhere Lernraten.
- `Dropout(0.25)` nach jedem Block und vor der Ausgabe verhindert Overfitting.
- `padding="same"` beim ersten Conv eines Blocks erhält die Auflösung, damit nach drei Pooling-Stufen noch 4×4 Feature-Maps übrig bleiben.

Danach `Flatten` $\rightarrow$ `Dense(128)` $\rightarrow$ `Dropout` $\rightarrow$ `Dense(10, softmax)`.

3. Kompilierung

```
optimizer = Adam(learning_rate=LEARNING_RATE) # 0.0005
model.compile(optimizer=optimizer, loss="sparse_categorical_crossentropy", metrics=["accuracy"])
```

4. Callbacks

```
checkpoint = ModelCheckpoint(MODEL_SAVE_PATH, monitor="val_accuracy", save_best_only=True, mode="max")
early_stopping = EarlyStopping(monitor="val_loss", patience=EARLY_STOPPING_PATIENCE)
lr_scheduler = LearningRateScheduler(lr_schedule)
```

- `ModelCheckpoint` schreibt nur, wenn `val_accuracy` einen neuen Bestwert erreicht – so liegt am Ende immer das beste Modell auf der Platte, auch wenn spätere Epochen schlechter werden.
- `EarlyStopping` bricht ab, wenn `val_loss` sich `patience` Epochen lang nicht verbessert.
- `lr_schedule` halbiert die Lernrate alle `STEP_DROP = 5` Epochen:

```
def lr_schedule(epoch, lr):
    if epoch % STEP_DROP == 0 and epoch:
        return lr * 0.5
    return lr
```

5. Training und Plot

`model.fit(..., epochs=50, batch_size=64, callbacks=[...])`. Die Lösung plottet `accuracy` und `val_accuracy` und speichert `plot.png` vor `plt.show()`.

6. Vorhersagemodus

```
model = load_model(MODEL_SAVE_PATH)
random_image = x_test[random.randint(0, len(x_test) - 1)]
prediction = model.predict(np.expand_dims(random_image, 0)) # Batch-Dimension
predicted_label = CIFAR10_LABELS[np.argmax(prediction)]
```

`expand_dims` ist nötig, weil das Modell immer einen Batch (n, 32, 32, 3) erwartet. `argmax` über die 10 Softmax-Ausgaben liefert die Klasse.

Ergebnis

Mit dieser Konfiguration erreicht das Modell auf den Testdaten je nach Trainingsdauer **ca. 88–91 %**. Auf CPU dauert das vollständige Training mit Augmentation mehrere Stunden; zum Ausprobieren `EPOCHS` reduzieren oder die Augmentation auskommentieren.